

STAGE 3

Setting paths:

SRC="/home/user01/ghidra-codeql-analysis/ghidra-10.0.4"

DB="/home/user01/ghidra-codeql-analysis/codeql-db-ghidra"

R="/home/user01/ghidra-codeql-analysis/reports"

Q="/home/user01/ghidra-codeql-analysis/codeql-ghidra-reachability"

mkdir -p "\$R" "\$Q/queries"

cd "\$Q"

cat > qlpack.yml <<'EOF'

name: local/ghidra-log4shell-queries

version: 0.0.1

dependencies:

codeql/java-all: "*"

EOF

codeql pack install

Create codeql database:

rm -rf "\$DB"

codeql database create "\$DB" \

--language=java \

--source-root "\$SRC" \

```
--build-mode=none
```

Query 1 direct calls to Jndilookup.lookup():

```
cat > "$Q/queries/10-direct-calls-to-jndilookup-lookup.ql" <<'EOF'
```

```
/**
```

```
 * @name Direct calls to JndiLookup.lookup
```

```
 * @description Finds direct calls from Ghidra source code to  
org.apache.logging.log4j.core.lookup.JndiLookup.lookup().
```

```
 * @kind problem
```

```
 * @problem.severity warning
```

```
 * @id local/direct-calls-to-jndilookup-lookup
```

```
 */
```

```
import java
```

```
predicate isGhidraCallable(Callable c) {
```

```
  c.fromSource() and
```

```
  exists(string q |
```

```
    q = c.getQualified_name() and
```

```
    (
```

```
      q.matches("ghidra.%") or
```

```
      q.matches("docking.%") or
```

```
      q.matches("generic.%") or
```

```
      q.matches("db.%")
```

```
    )
```

```
  )
```

```
}
```

```
predicate isJndiLookupLookup(Callable c) {  
    c.hasQualifiedName(  
        "org.apache.logging.log4j.core.lookup",  
        "JndiLookup",  
        "lookup"  
    )  
}
```

```
from Call call, Callable caller, Callable target  
where  
    call.getCaller() = caller and  
    call.getCallee() = target and  
    isGhidraCallable(caller) and  
    isJndiLookupLookup(target)  
select call,  
    "Direct call from " + caller.getQualifiedName() +  
    " to " + target.getQualifiedName()  
EOF
```

```
codeql query run "$Q/queries/10-direct-calls-to-jndillookup-lookup.ql" \  
    --database="$DB" \  
    --output="$R/10-direct-calls-to-jndillookup-lookup.bqrs"
```

```
codeql bqrs decode \  
    --format=csv \  

```

```
--output="$R/10-direct-calls-to-jndillookup-lookup.csv" \  
"$R/10-direct-calls-to-jndillookup-lookup.bqrs"
```

```
cat "$R/10-direct-calls-to-jndillookup-lookup.csv"
```

Query2 direct references to Jndillookup:

```
cat > "$Q/queries/11-direct-type-references-to-jndillookup.ql" <<'EOF'
```

```
/**
```

```
 * @name Direct type references to JndiLookup
```

```
 * @description Finds direct source-code type references to  
org.apache.logging.log4j.core.lookup.JndiLookup in Ghidra code.
```

```
 * @kind problem
```

```
 * @problem.severity warning
```

```
 * @id local/direct-type-references-to-jndillookup
```

```
 */
```

```
import java
```

```
predicate isJndiLookupType(RefType t) {  
    t.hasQualifiedName(  
        "org.apache.logging.log4j.core.lookup",  
        "JndiLookup"  
    )  
}
```

```

predicate isInGhidraSource(TypeAccess ta) {
    ta.fromSource() and
    exists(string path |
        path = ta.getFile().getRelativePath() and
        (
            path.matches("%/Ghidra/%") or
            path.matches("Ghidra/%")
        )
    )
}

```

```

from TypeAccess ta, RefType t
where
    ta.getType() = t and
    isJndiLookupType(t) and
    isInGhidraSource(ta)
select ta,
    "Direct type reference to " + t.getQualifiedName()
EOF

```

```

codeql query run "$Q/queries/11-direct-type-references-to-jndillookup.ql" \
    --database="$DB" \
    --output="$R/11-direct-type-references-to-jndillookup.bqrs"

```

```

codeql bqrs decode \
    --format=csv \

```

```
--output="$R/11-direct-type-references-to-jndillookup.csv" \  
"$R/11-direct-type-references-to-jndillookup.bqrs"
```

```
cat "$R/11-direct-type-references-to-jndillookup.csv"
```